软件运行时复生与可复现基础设施

ReviveOS

把旧项目的运行环境找回来，把修复过程留下来。

代码还在，依赖、构建脚本和运行环境却已经失效。ReviveOS 从仓库文件里还原当年的环境，在 Docker 沙箱中完成构建和测试；模型可以提出补丁，但补丁必须经过执行验证。恢复结束后，环境、补丁、行为记录和 SBOM 会一起写进 `.revive` 胶囊。

环境考古 | 沙箱构建 | 受限修复 | 行为验证 | 复现胶囊

产品形态
CLI、Web 控制台和 Electron 桌面端共用一套后端：FastAPI + PostgreSQL + Redis + S3，执行由 Celery worker 和 Docker 沙箱完成。

适用范围

当前重点处理 Python 项目，覆盖 2.7 至 3.12。依赖清单、`setup.py`、`pyproject.toml`、Pipfile、conda 环境、Dockerfile 和 CI 配置都可以作为环境证据。

输入

项目仓库、目标版本，以及可用的测试或行为基线。

输出

环境推断、构建与测试记录、补丁、评分和可校验的复现胶囊。

旧项目缺的，往往不是代码。

项目自带的 `py27-flask` 样例记录了一个典型过程：2015 年使用 Flask 0.10，2016 年升级到 0.12，运行环境固定在 Python 2.7。今天打开它，依赖源、系统库和构建脚本都可能已经变化，直接安装大概率失败。

ReviveOS 先回答三个问题：当年它运行在什么环境里，现在还能不能构建，构建出来之后行为是否还和原来一致。

问题	常见表现	处理方式
环境线索分散	依赖清单、CI、Dockerfile 和 README 各说各话。	统一提取，保留来源、文件和行号。
依赖版本漂移	锁文件缺失，旧版本从索引消失。	锁文件和精确版本优先，再查历史发布；无法确认的依赖保留 `*`。
修复没有边界	补丁可能改测试、改依赖，越改越大。	限制文件数和行数，禁止修改测试，失败回滚。
结果无法交接	过程留在个人终端和聊天记录里。	manifest、依赖、补丁、行为记录和 SBOM 一起归档。

适用场景

开源项目接管

接手停更项目，先摸清原始环境，再判断修复范围。

科研代码复现

保存当年的依赖和运行结果，方便论文复现。

企业遗留工具

迁移旧脚本前，先建立可运行基线。

教学与归档

用真实项目演示依赖漂移，留下可检查的实验记录。

> 一个恢复结果要能回答：环境是怎么推断出来的，补丁改了什么，测试跑了什么，哪些信息仍然缺失。

一次恢复，六个阶段。

01 建立任务

选择仓库，设置预算和超时。用户需要拥有代码分析和运行授权。

02 环境考古

读取 requirements、setup.py、pyproject、Pipfile、conda、Dockerfile、CI 和 README，输出 Python 版本、操作系统、依赖和置信度。py27-flask 的结果是 Python 2.7、Debian、0.786。[1]

03 构建环境

按 Python 2.7 至 3.12 的镜像矩阵选择基础环境。容器内构建，运行时禁网、非 root、只读根文件系统、tmpfs，并限制 CPU、内存和进程数。[2]

04 修复

构建失败时，模型提出统一 diff。单次最多 10 个文件、300 行增删，不能修改测试；最多 5 次构建、4 次修复；失败恢复上一版快照。[3]

05 行为验证

采集退出码、stdout 哈希、指定文件、数值序列和 JSON 结构。有基线时比较，没有基线时记录缺失。[4]

06 生成胶囊

写入 manifest、genome、dependencies.lock、Dockerfile、patches、行为计划与指纹、SBOM、provenance 和评分，使用确定性 tar.gz，附 source.tar。[5]

三个入口

入口	用途
Web 控制台	提交任务、观察阶段事件、查看报告与胶囊，进行系统设置。
CLI	analyze、resurrect、verify、replay，支持脚本化调用。
桌面端	Electron 壳，连接本地或远端后端。

> 任务成功、评分和胶囊是三份不同的记录：任务成功表示流程跑完；评分记录各项工程指标；胶囊保存这次恢复的材料。

控制面与执行面分开。

API 不执行用户代码。构建、测试和行为采集都交给 Celery worker，再进入 Docker 沙箱。API 可以独立扩容，worker 可以部署在隔离主机上。

浏览器 / Electron / CLI
提交任务、接收事件、查看证据、下载胶囊

FastAPI 控制面 • REST / SSE
认证、项目与任务、配额、审计、运行配置

PostgreSQL + Redis + S3
关系数据 / 事件与限流 / 胶囊对象存储

Celery worker + LangGraph
阶段编排、模型调用、预算控制、结果汇总

核心引擎 + Docker 沙箱
环境考古、构建与修复、行为采集、评分与胶囊

层次	技术	职责
交互层	Next.js / TypeScript / Electron	Web 与桌面交互，展示服务端结果。
控制层	FastAPI / SQLAlchemy / Alembic	API、认证、持久化与数据迁移。
调度层	Celery / Redis / LangGraph	异步任务、事件和智能体流程。
执行层	Python 核心引擎 / Docker	证据解析、构建、修复和行为验证。
存储与交付	PostgreSQL / S3 / JSON Schema	结构化数据、胶囊正文和格式契约。

生产配置

- 启动：`REVIVE_ENV=prod` 时，缺少 `DATABASE_URL`、`REDIS_URL`、S3 配置或持久化适配器会直接拒绝启动。
- 认证：JWT access + refresh，刷新令牌轮换并检测重用；API Key 哈希存储，默认 90 天过期。
- 运行：`/healthz` 存活检查，`/readyz` 检查 PostgreSQL、Redis 和对象存储，`/metrics` 输出 Prometheus 指标；Caddy 终止 TLS，只暴露 80/443。

模型接入

支持 fake、OpenAI、Anthropic 和 custom 提供方；管理员配置模型名、Base URL、密钥和推理强度；每个任务有预算和调用次数上限。[6][7]

把模型放进一条可检查的工程链里。

01 先有证据，再有建议

依赖文件、项目配置和构建线索会先被解析成带来源的结构化证据。确定性解析不依赖模型，也能独立产出环境基因；模型只补充信息，不负责凭空补版本。无法确认的依赖保留 *，置信度随证据冲突下降。

02 修复有硬边界

补丁按实际 diff 计数，单次最多 10 个文件、300 行增删；只能修改工作区清单里的文件，测试文件禁止改动。构建失败最多重试 5 次，对应最多 4 次修复调用；每次应用补丁前保留快照，失败后恢复上一版。

03 看行为，不只看构建

构建通过只说明代码能编译或安装。ReviveOS 还会采集退出码、stdout 哈希、指定文件、数值序列和 JSON 结构；有基线时逐项比较，没有基线时把缺失记录下来，而不是默认通过。

04 胶囊保存恢复材料

一个胶囊通常包含 14 个条目：manifest、genome、依赖记录、Dockerfile、补丁、行为计划与指纹、SBOM、provenance 和评分。归档使用确定性 tar.gz，同样的输入会得到同样的目录结构，方便校验和转交。

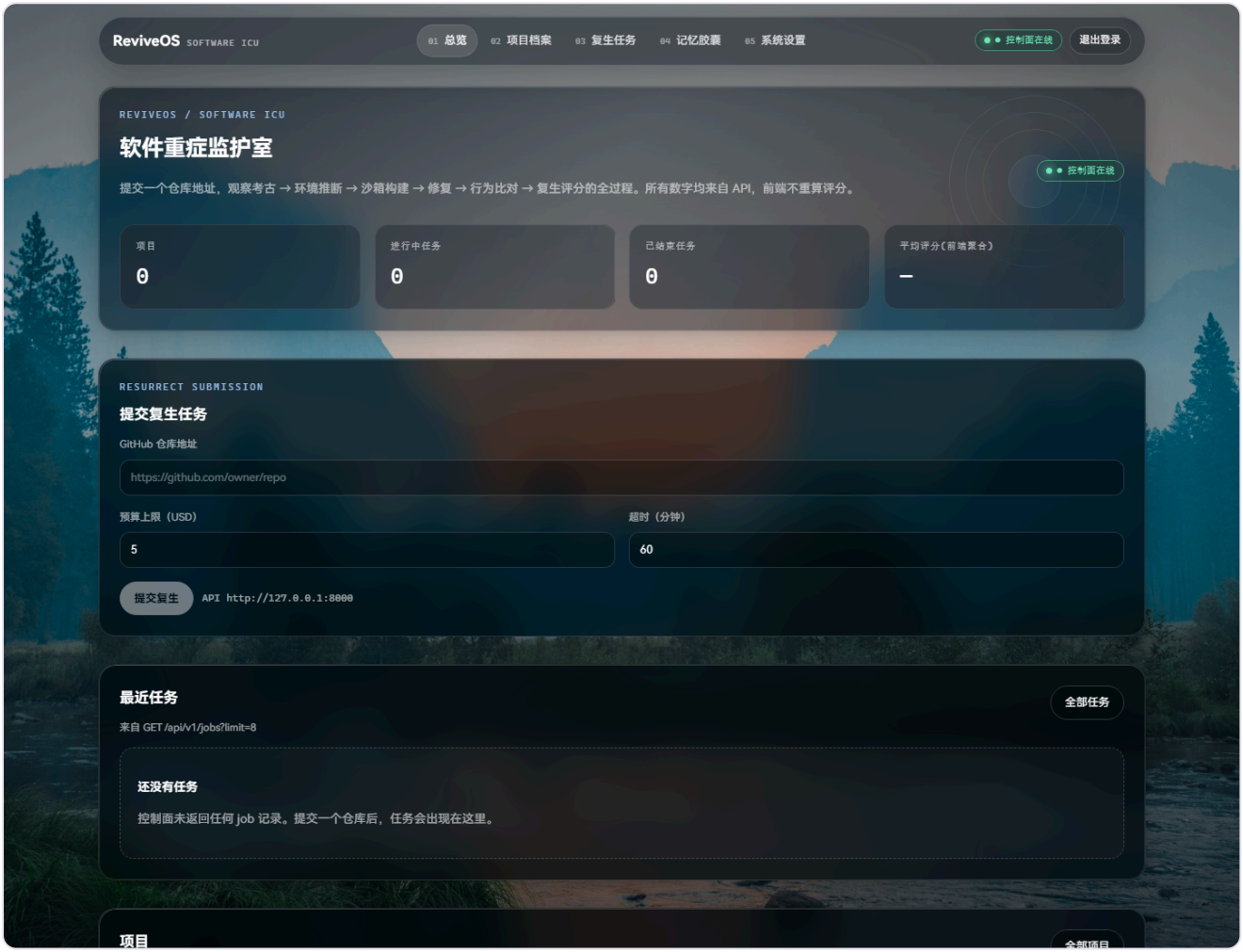
05 任务、预算和审计都在控制面

模型调用、异步任务、事件、配额、API Key、审计日志和 readiness 检查统一由后端管理。管理员可以按任务设置预算，worker 在隔离主机上执行用户代码。

常见工具	ReviveOS 的做法
包管理器：安装已知依赖	先从碎片化线索推断环境，再进入构建和行为验证。
容器工具：复用已知环境	生成环境描述，并把恢复材料封装成可校验胶囊。
代码助手：生成修改建议	把建议放进构建、测试、回滚和留证流程。

控制台与演示片。

控制台围绕一次恢复任务组织：提交项目、查看阶段事件、检查证据、阅读报告、下载胶囊。设置页可以更换壁纸，也可以配置模型提供方。



实际界面截图，画面为空任务状态。

四分钟发布片

发布片按使用顺序展开：01 提交仓库、预算和超时 → 02 查看环境推断 → 03 观察任务事件 → 04 下载胶囊，最后补充 CLI 用法。画面由真实控制台截图、真实分析结果和代码绘制图形组成。

> **发布片**
> 4 分钟，1920 × 1080，H.264 / AAC，含配乐。另有一段 80 秒宣传片，用于展台循环播放。

上线前，按这张表走。

当前版本已完成生产部署验收。生产环境使用 prod Compose：Caddy、API、worker、beat、PostgreSQL、Redis 和 S3。[8] 迁移先执行，API 通过 readiness 检查后接流量。

步骤	要做的事
环境文件	从 `infra/envs/revive.prod.env.template` 生成生产配置。JWT 密钥、worker token、数据库和 Redis 口令、S3 凭据交给 secret manager 管理；LLM 密钥可以在管理端填写。
迁移与管理员	执行 `python -m alembic -c apps/api/alembic.ini upgrade head`。首个管理员先通过 `/login` 注册，再在数据库中把角色改为 admin。
启动与探针	`docker compose -f infra/compose/prod.docker-compose.yml up -d`。检查 `/healthz` 和 `/readyz`；readiness 会检查 PostgreSQL、Redis 和对象存储。
网络、隔离与备份	Caddy 负责 80/443 和证书。worker 单独部署在隔离主机，因为 Docker socket 等同于宿主 root。备份使用 `infra/backup/pg-backup.sh` 与 `pg-restore.sh`，S3 开启版本控制并定期演练恢复。
发布与运行	tag 触发 GHCR 构建、Trivy 扫描和 CycloneDX SBOM，基础镜像 digest 固定后上线。`/metrics` 接入 Prometheus，设置任务预算、并发配额和登录限流。

验证记录

检查	结果
自动化测试	`pytest -q`：449 项通过，40.49 秒（2026-10-28）
契约与静态检查	OpenAPI 快照与运行时一致；镜像摘要和红线规则通过
CI 配置	质量门禁、三镜像构建、API `/healthz` 与 Web 首页冒烟

代码依据

[1]	`packages/core/src/revive_core/archaeology/extractors/__init__.py:37`
[2]	`packages/core/src/revive_core/sandbox/port_impl.py:552`
[3]	`packages/core/src/revive_core/sandbox/patch_flow.py:200`
[4]	`packages/core/src/revive_core/behavior/capture.py:357`
[5]	`packages/core/src/revive_core/capsule/builder.py:119`
[6]	`apps/api/src/revive_api/tasks/pipeline.py:173`
[7]	`agents/src/revive_agents/llm/factory.py:76`
[8]	`infra/compose/prod.docker-compose.yml:4`